

CHAPTER 3- MEMORY MANAGEMENT

Memory is an important resource that must be carefully managed. Even though the amount of memory used in computers are increasing remarkably, programs are getting bigger faster than memories - programs expand to fill the memory available to hold them.

Ideally, what every programmer would like is an infinitely large, inexpensive, infinitely fast memory that is also nonvolatile. Unfortunately technology does not provide such memories. Consequently, most computers have a **memory hierarchy**, with a small amount of very fast, expensive, volatile cache memory, few gigabytes of medium-speed, medium-price, volatile main memory (RAM), and hundreds of gigabytes of slow, cheap, nonvolatile disk storage. It is the job of the operating system to coordinate how these memories are used.

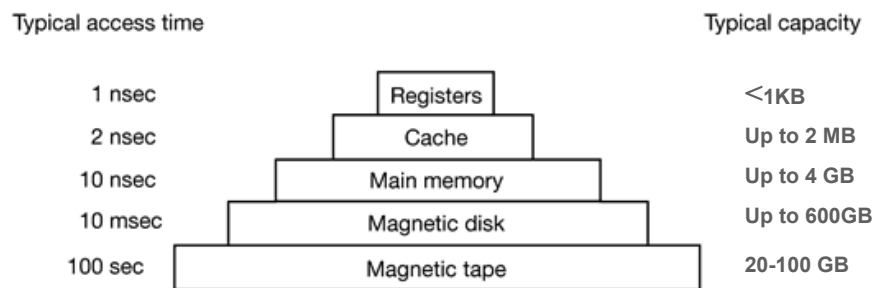


Figure 3.1 A typical memory hierarchy (the numbers are for PCs and very rough approximations)

The part of the operating system that manages the memory hierarchy is called the **memory manager**. Its job is:

- to keep track of which parts of memory are in use
- to keep track of which parts are not in use
- to allocate memory to processes when they need it
- to deallocate memory when processes are done and
- to manage swapping between main memory and disk when main memory is too small to hold all the processes.

In this chapter we will investigate a number of different memory management schemes, ranging from very simple to highly sophisticated. We will start at the beginning and look first at the simplest possible memory management system and then gradually progress to more and more elaborate ones.

Basic Memory Management

Memory management systems can be divided into two classes: *those that move processes back and forth between main memory and disk during execution (swapping and paging), and those that do not*. The latter are simpler, so we will study them first.

Memory Management Schemes that do not SWAP processes

Monoprogramming without Swapping or Paging- is the simplest possible memory management scheme that runs just one program at a time, sharing the memory between that program and the operating system. When the system is organized in this way, only one process at a time can be running. As soon as the user types a command, the operating system copies the requested program from disk to memory and executes it. When the process finishes, the operating system displays a prompt character and waits for a new command. When it receives the command, it loads a new program into memory, overwriting the first one. Except on simple embedded systems, monoprogramming is hardly used any more.

Multiprogramming with Fixed Partitions - systems allow multiple processes to run at the same time. Having multiple processes running at once means that when one process is blocked waiting for I/O to finish, another one can use the CPU. Thus multiprogramming increases the CPU utilization. This scheme achieves multiprogramming by simply dividing memory up into n (possibly unequal) partitions. This partitioning can, for example, be done manually when the system is started up.

When a job arrives, it can be put into the input queue for the smallest partition large enough to hold it. Since the partition sizes are fixed in this scheme, any space in a partition not used by a job is lost and this is known as **internal fragmentation**.

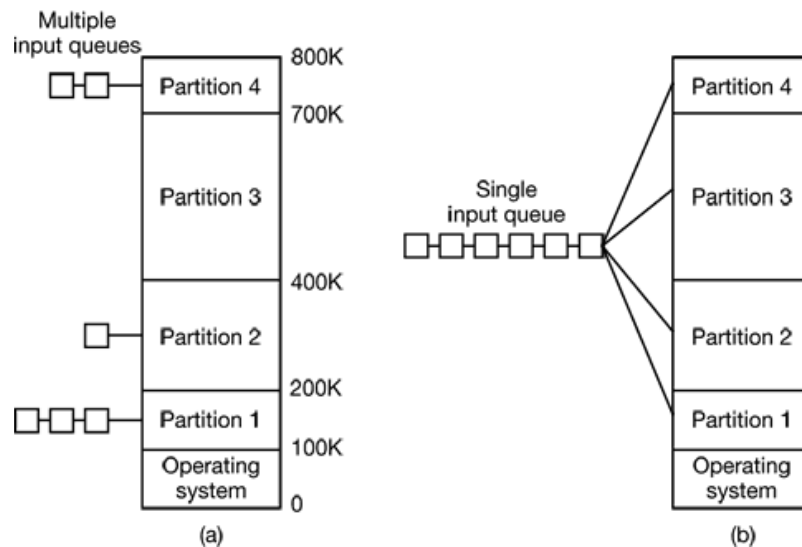


Figure 3-2. (a) Fixed memory partitions with separate input queues for each partition
(b) Fixed memory partitions with a single input queue

This system, with fixed partitions set up by the operator in the morning and not changed thereafter, was used by OS/360 on large IBM mainframes for many years. It was called **MFT** (Multiprogramming with a Fixed number of Tasks or OS/MFT). Nowadays, few, if any, operating systems support this model because of its several limitations listed below.

- Since partition sizes are fixed, larger programs should be split the program into pieces, called **overlays**. Overlay 0 would start running first. When it was done, it would call another

overlay. This work of splitting the program into pieces had to be done by the programmer and was time consuming and boring.

- The degree of multiprogramming (number of active process in memory) is limited by the number of slots defined.
- The degree of internal fragmentation is high

Multiprogramming with Dynamic Partitions- is a scheme in which the number and size of partitions is variable. When a process is brought in to main memory, it is allocated exactly as much memory as it requires and no more. This scheme starts out well, but eventually it leads to a situation in which there are a lot of small holes in memory. As time goes on , memory becomes more and more fragmented and memory utilization declines. This phenomenon is referred to as ***External fragmentation*** referring to the fact that the memory that is external to all partitions becomes increasingly fragmented. One solution to overcome this problem of external fragmentation is ***compaction***. In this technique, the OS ,from time to time ,shifts the processes so that so that they are contiguous and so that all of the free memory is together in one block. But this task is quite time consuming.

Fig. 3.3 The effect of dynamic Partition

Buddy systems

Managing Free/Allocated Memory

When memory is assigned dynamically, the operating system must manage it. In general terms, there are two ways to keep track of memory usage: bitmaps and linked lists.

- **Memory Management with Bitmaps** -With a bitmap, memory is divided up into allocation units, perhaps as small as a few words and perhaps as large as several kilobytes. Corresponding to each allocation unit is a bit in the bitmap, which is 0 if the unit is free and 1 if it is occupied (or vice versa). Figure 3.4 shows part of memory and the corresponding bitmap.

A bitmap provides a simple way to keep track of memory words in a fixed amount of memory because the size of the bitmap depends only on the size of memory and the size of the allocation unit. The main problem with it is that when it has been decided to bring a k unit process into memory, the memory manager must search the bitmap to find a run of k consecutive 0 bits in the map. Searching a bitmap for a run of a given length is a slow operation (because the run may straddle word boundaries in the map).

- **Memory Management with Linked Lists**- in this scheme the OS maintains a linked list of allocated and free memory segments, where a segment is either a process or a hole between two processes. The memory of Fig. 3.4(a) is represented in Fig. 3.4(c) as a linked list of segments. Each entry in the list specifies a hole (H) or process (P), the address at which it starts, the length, and a pointer to the next entry.

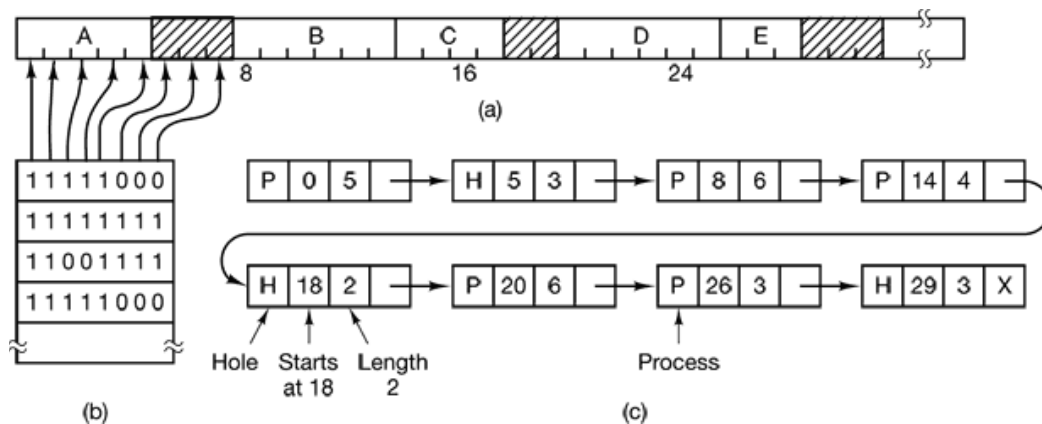


Figure 3.4 (a) A part of memory with five processes and three holes. The tick marks show the memory allocation units. The shaded regions (0 in the bitmap) are free. (b) The corresponding bitmap. (c) The same information as a list.

Relocation and Protection

Multiprogramming introduces two essential problems that must be solved—relocation and protection. Look at Fig. 3-2. From the figure it is clear that different jobs will be run at different addresses. When a program is linked (i.e., the main program, user-written procedures, and library procedures are combined into a single address space), the linker must know at what address the program will begin in memory.

For example, suppose that the first instruction is a call to a procedure at absolute address 100 within the binary file produced by the linker. If this program is loaded in partition 1 (at address 100K), that instruction will jump to absolute address 100, which is inside the operating system. What is needed is a call to $100K + 100$. If the program is loaded into partition 2, it must be carried out as a call to $200K + 100$, and so on. This problem is known as the relocation problem.

One possible solution is to actually modify the instructions as the program is loaded into memory. Programs loaded into partition 1 have 100K added to each address, programs loaded into partition 2 have 200K added to addresses, and so forth. To perform relocation during loading like this, the linker must include in the binary program a list or bitmap telling which program words are addresses to be relocated and which are opcodes, constants, or other items that must not be relocated. OS/MFT worked this way.

In multiuser systems, it is highly undesirable to let processes read and write memory belonging to other users. The solution that IBM chose for protecting the 360 was to divide memory into blocks of 2-KB bytes and assign a 4-bit protection code to each block. The PSW (Program Status Word) contained a 4-bit key. The 360 hardware trapped any attempt by a running process to access memory whose protection code differed from the PSW key. Since only the operating system could change the protection codes and key, user processes were prevented from interfering with one another and with the operating system itself.

Memory Management Schemes that SWAP processes

With a batch system, organizing memory into fixed partitions is simple and effective. Each job is loaded into a partition when it gets to the head of the queue. It stays in memory until it has finished. As long as enough jobs can be kept in memory to keep the CPU busy all the time, there is no reason to use anything more complicated.

With timesharing systems or graphically oriented personal computers, the situation is different. Sometimes there is not enough main memory to hold all the currently active processes, so excess processes must be kept on disk and brought in to run dynamically.

Two general approaches to memory management can be used, depending (in part) on the available hardware. The simplest strategy, called **swapping**, consists of bringing in each process in its entirety, running it for a while, then putting it back on the disk. The other strategy, called **virtual memory**, allows programs to run even when they are only partially in main memory.

The operation of a swapping system is illustrated in Fig. 3.5. Initially only process *A* is in memory. Then processes *B* and *C* are created or swapped in from disk. In Fig. 3-5(d) *A* is swapped out to disk. Then *D* comes in and *B* goes out. Finally *A* comes in again.

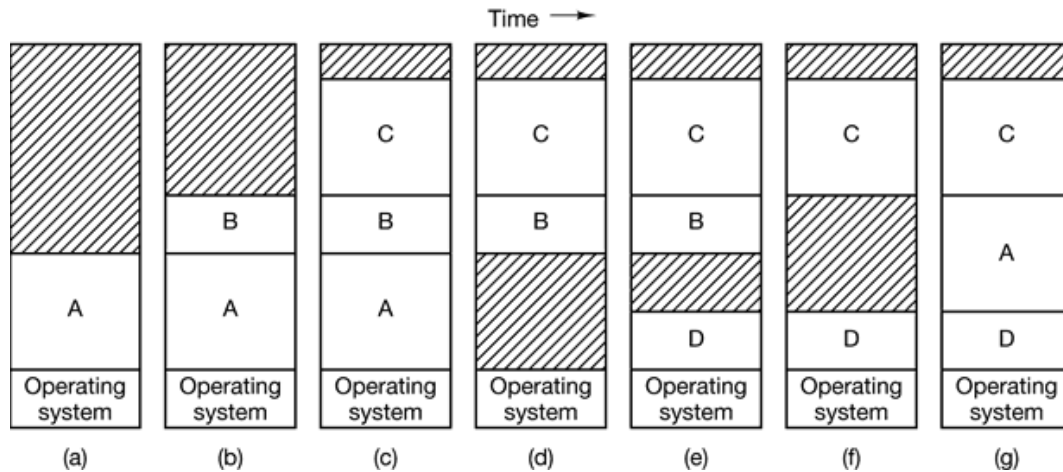


Figure 3.5 - Memory allocation changes as processes come into memory and leave it (the shaded regions are unused memory).

When swapping creates multiple holes in memory, it is possible to combine them all into one big one by moving all the processes downward as far as possible. This technique is known as **memory compaction**. It is usually not done because it requires a lot of CPU time. For example, on a 256-MB machine that can copy 4 bytes in 40 nsec, it takes about 2.7 sec to compact all of memory.

VIRTUAL MEMORY

Many years ago people were first confronted with programs that were too big to fit in the available memory. The solution usually adopted was to split the program into pieces, called **overlays**. Overlay 0 would start running first. When it was done, it would call another overlay. Some overlay systems were highly complex, allowing multiple overlays in memory at once. The overlays were kept on the disk and swapped in and out of memory by the operating system, dynamically, as needed.

Although the actual work of swapping overlays in and out was done by the system, the work of splitting the program into pieces had to be done by the programmer. Splitting up large programs into small, modular pieces was time consuming and boring. It did not take long before someone thought of a way to turn the whole job over to the computer. The method that was devised has come to be known as **virtual memory**.

The basic idea behind virtual memory is that the combined size of the program, data, and stack may exceed the amount of physical memory available for it. The operating system keeps those parts of the program currently in use in main memory, and the rest on the disk. For example, a 16-MB program can run on a 4-MB machine by carefully choosing which 4 MB to keep in memory at each instant, with pieces of the program being swapped between disk and memory as needed. Virtual memory is based on one of the two basic techniques: **Segmentation and Paging**.

Paging

Most virtual memory systems use a technique called **paging**. On any computer, there exists a set of memory addresses that programs can produce. When a program uses an instruction like

```
MOV REG, 1000
```

it does this to copy the contents of memory address 1000 to REG (or vice versa, depending on the computer). Addresses can be generated using indexing, base registers, segment registers, and other ways.

These program-generated addresses are called **virtual addresses** and form the **virtual address space**. On computers without virtual memory, the virtual address is put directly onto the memory bus and causes the physical memory word with the same address to be read or written. When virtual memory is used, the virtual addresses do not go directly to the memory bus. Instead, they go to an **MMU (Memory Management Unit)** that maps the virtual addresses onto the physical memory addresses as illustrated in Fig. 3.6.

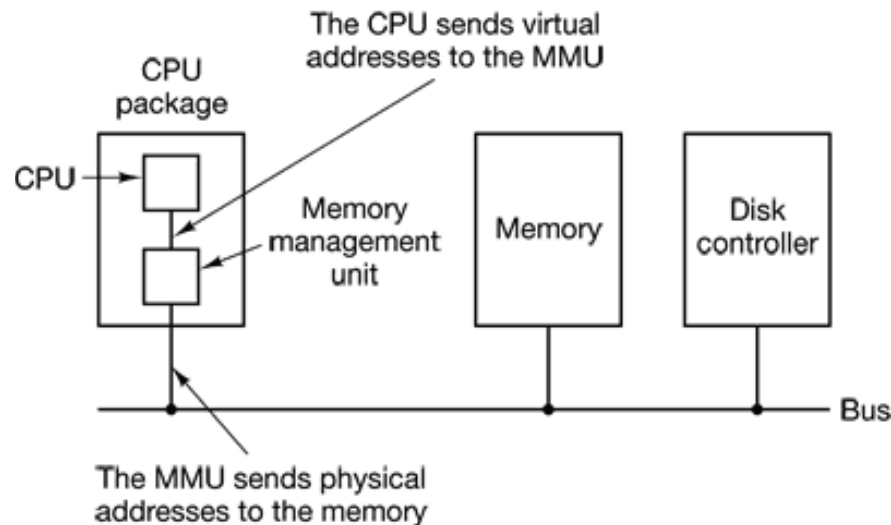


Figure 3.6. The position and function of the MMU

A very simple example of how this mapping works is shown in Fig. 3.7. In this example, we have a computer that can generate 16-bit addresses, from 0 up to 64K. These are the virtual addresses. This computer, however, has only 32 KB of physical memory, so although 64-KB programs can be written, they cannot be loaded into memory in their entirety and run. A complete copy of a program's core image, up to 64 KB, must be present on the disk, however, so that pieces can be brought in as needed.

The virtual address space is divided up into units called **pages**. The corresponding units in the physical memory are called **page frames**. The pages and page frames are always the same size. In this example they are 4 KB, but page sizes from 512 bytes to 64 KB have been used in real systems. With 64 KB of virtual address space and 32 KB of physical memory, we get 16 virtual pages and 8 page frames. Transfers between RAM and disk are always in units of a page.

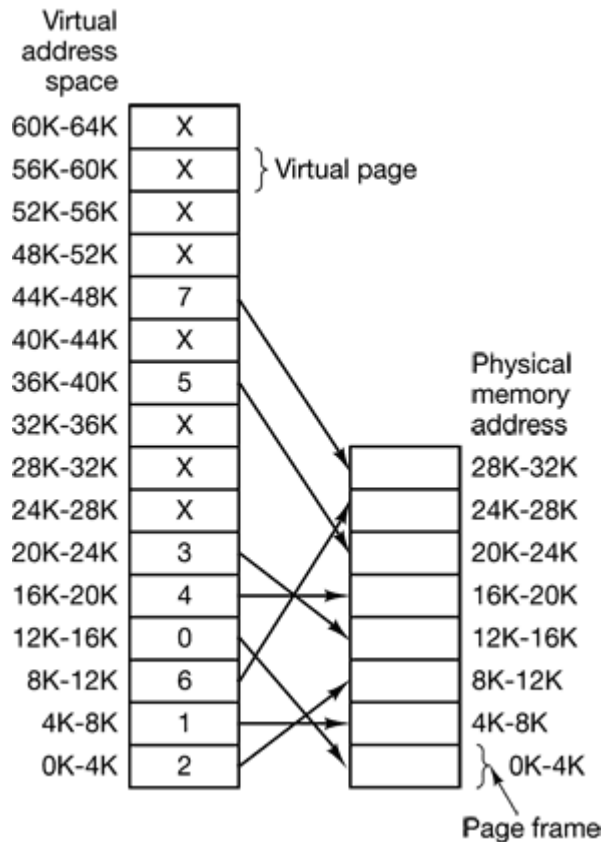


Figure 3.7 The relation between virtual addresses and physical memory addresses is given by the page table

When the program tries to access address 0, for example, using the instruction `MOV REG,0` virtual address 0 is sent to the MMU. The MMU sees that this virtual address falls in page 0 (0 to 4095), which according to its mapping is page frame 2 (8192 to 12287). It thus transforms the address to 8192 and outputs address 8192 onto the bus. The memory knows nothing at all about the MMU and just sees a request for reading or writing address 8192, which it honors. Thus, the MMU has effectively mapped all virtual addresses between 0 and 4095 onto physical addresses 8192 to 12287. Similarly, an instruction `MOV REG, 8192` is effectively transformed into `MOV REG, 24576` because virtual address 8192 is in virtual page 2 and this page is mapped onto physical page frame 6 (physical addresses 24576 to 28671).

When a program tries to use an unmapped page, for example, by using the instruction

`MOV REG, 32780`

which is byte 12 within virtual page 8 (starting at 32768), the MMU notices that the page is unmapped (indicated by a cross in the figure) and causes the CPU to trap to the operating system. This trap is called a **page fault**. The operating system picks a little-used page frame and writes its contents back to the disk. It then fetches the page just referenced into the page frame just freed, changes the map, and restarts the trapped instruction.

For example, if the operating system decided to evict page frame 1, it would load virtual page 8 at physical address 4K and make two changes to the MMU map. First, it would mark virtual page 1's entry as unmapped, to trap any future accesses to virtual addresses between 4K and 8K, Then it

would replace the cross in virtual page 8's entry with a 1, so that when the trapped instruction is reexecuted, it will map virtual address 32780 onto physical address 4108.

Page Tables

The purpose of the page table is to map virtual pages onto page frames. Mathematically speaking, the page table is a function, with the virtual page number as argument and the physical frame number as result. Using the result of this function, the virtual page field in a virtual address can be replaced by a page frame field, thus forming a physical memory address.

The virtual page number is used as an index into the page table to find the entry for that virtual page. From the page table entry, the page frame number (if any) is found. The page frame number is attached to the high-order end of the offset, replacing the virtual page number, to form a physical address that can be sent to the memory.

Despite this simple description, two major issues must be faced:

- *the page table can be extremely large*
- *the mapping must be fast.*

The first point follows from the fact that modern computers use virtual addresses of at least 32 bits. With, say, a 4-KB page size, a 32-bit address space has 1 million pages, and a 64-bit address space has more than you want to contemplate. With 1 million pages in the virtual address space, the page table must have 1 million entries. And remember that each process needs its own page table (because it has its own virtual address space).

The second point is a consequence of the fact that the virtual-to-physical mapping must be done on every memory reference. A typical instruction has an instruction word, and often a memory operand as well. Consequently, it is necessary to make 1, 2, or sometimes more page table references per instruction. If an instruction takes, say, 4 nsec, the page table lookup must be done in under 1 nsec to avoid becoming a major bottleneck.

The exact layout of an entry is highly machine dependent, but the kind of information present is roughly the same as shown in Figure 3-8. The most important field is the Page frame number. After all, the goal of the page mapping is to locate this value.

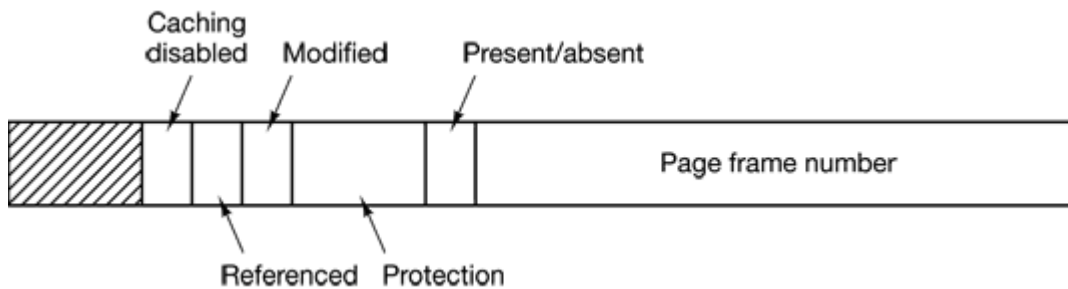


Figure 3.8 A typical page table entry

- **Present/absent bit** shows presence. If this bit is 1, the entry is valid and can be used. If it is 0, the virtual page to which the entry belongs is not currently in memory. Accessing a page table entry with this bit set to 0 causes a page fault.
- **Protection bits** tell what kinds of access are permitted. In the simplest form, this field contains 1 bit, with 0 for read/write and 1 for read only.
- **Modified bit** keep track of page usage. When a page is written to, the hardware automatically sets the Modified bit. This bit is of value when the operating system decides to reclaim a page frame. If the page in it has been modified (i.e., is “dirty”), it must be written back to the disk. If it has not been modified (i.e., is “clean”), it can just be abandoned, since the disk copy is still valid. The bit is sometimes called the **dirty bit**, since it reflects the page’s state.
- **Referenced bit** is set whenever a page is referenced, either for reading or writing. Its value is to help the operating system choose a page to evict when a page fault occurs. Pages that are not being used are better candidates than pages that are, and this bit plays an important role in several of the page replacement algorithms that we will study later in this section.
- **Caching disabled field** - allows caching to be disabled for the page. This feature is important for pages that map onto device registers rather than memory. If the operating system is sitting in a tight loop waiting for some I/O device to respond to a command it was just given, it is essential that the hardware keeps fetching the word from the device, and not use an old cached copy. With this bit, caching can be turned off.

TLB (Translation Lookaside Buffers)

In most paging schemes, the page tables are kept in memory. Due to their large size, this design has an enormous impact on performance. Computer designers have known about this problem for years and have come up with a solution. Their solution is based on the observation that most programs tend to make a large number of references to a small number of pages, and not the other way around. Thus only a small fraction of the page table entries are heavily read; the rest are barely used at all. Therefore, we can equip computers with a small hardware device for mapping virtual addresses to physical addresses without going through the page table. The device, called a **TLB (Translation Lookaside Buffer)** or sometimes an **associative memory**, is illustrated in Table 3-1.

It is usually inside the MMU and consists of a small number of entries and each entry contains information about one page, including the virtual page number, a bit that is set when the page is modified, the protection code (read/write/execute permissions), and the physical page frame in which the page is located. These fields have a one-to-one correspondence with the fields in the page table. Another bit indicates whether the entry is valid (i.e., in use) or not.

Table 3-1 A TLB to speed up paging

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

Inverted Page Tables

Traditional page tables of the type described so far require one entry per virtual page, since they are indexed by virtual page number. If the address space consists of 2^{32} bytes, with 4096 bytes per page, then over 1 million page table entries are needed. For 64-bit computers, we need a page table with 2^{52} entries and if each entry is 8 bytes, the table is over 30 million gigabytes. One solution to such situations is using **inverted page table**. In this design, there is one entry per page frame in real memory, rather than one entry per page of virtual address space. For example, with 64-bit virtual addresses, a 4-KB page, and 256 MB of RAM, an inverted page table only requires 65,536 entries. The entry keeps track of which (process, virtual page) is located in the page frame and can be created by using a hash function.

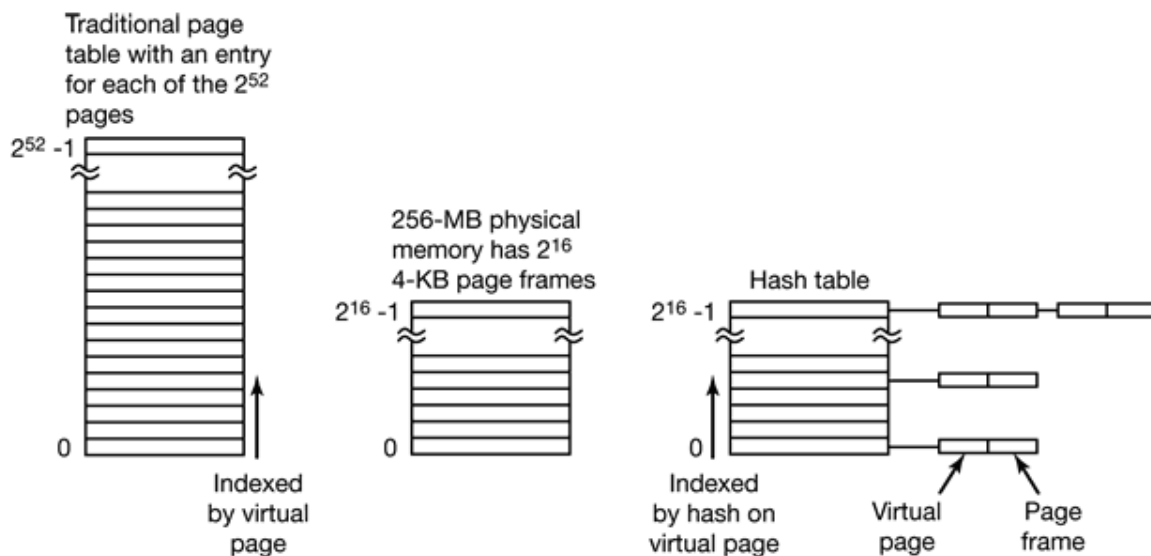


Figure 3-9 Comparison of a traditional page table with an inverted page table

Page Replacement

When a page fault occurs, the operating system has to choose a page to remove from memory to make room for the page that has to be brought in. If the page to be removed has been modified

while in memory, it must be rewritten to the disk to bring the disk copy up to date. If, however, the page has not been changed (e.g., it contains program text), the disk copy is already up to date, so no rewrite is needed. The page to be read in just overwrites the page being evicted.

While it would be possible to pick a random page to evict at each page fault, system performance is much better if a page that is not heavily used is chosen. If a heavily used page is removed, it will probably have to be brought back in quickly, resulting in extra overhead. Much work has been done on the subject of page replacement algorithms, both theoretical and experimental. Below we will describe some of the most important algorithms.

- **Optimal Page Replacement Algorithm** -Each page can be labeled with the number of instructions that will be executed before that page is first referenced and this algorithm simply removes that the page with the highest label should be removed. If one page will not be used for 8 million instructions and another page will not be used for 6 million instructions, the first one will be removed.

Limitation

- The operating system has no way of knowing when each of the pages will be referenced next at the time of the page fault

- **Not Recently Used Page Replacement Algorithm** – removes the page that has not been recently used by looking in to the referenced and modified bits in the page table entry.
- **First-In, First-Out (FIFO) Page Replacement Algorithm** - the operating system maintains a list of all pages currently in memory, with the page at the head of the list the oldest one and the page at the tail the most recent arrival.

Limitation:

- It may remove an important and heavily used process
- **Second Chance Page Replacement Algorithm** – looks for an old page that has not been referenced in the previous clock interval and replaces that page. It is a simple modification to FIFO that avoids the problem of throwing out a heavily used page. It inspects the *referenced* bit of the oldest page and if it is 0, the page is both old and unused, so it is replaced immediately. If the *referenced* bit is 1, the bit is cleared, the page is put onto the end of the list of pages, and its load time is updated as though it had just arrived in memory.
- **Clock Page Replacement Algorithm** –keeps all the page frames on a circular list in the form of a clock, as shown in Fig. 3-10. When a page fault occurs, the page being pointed to by the hand is inspected and if its *referenced* bit is 0, the page is evicted, the new page is inserted into the clock in its place, and the hand is advanced one position. If *referenced* bit is 1, it is cleared and the hand is advanced to the next page. This process is repeated until a page with $R = 0$ is found.

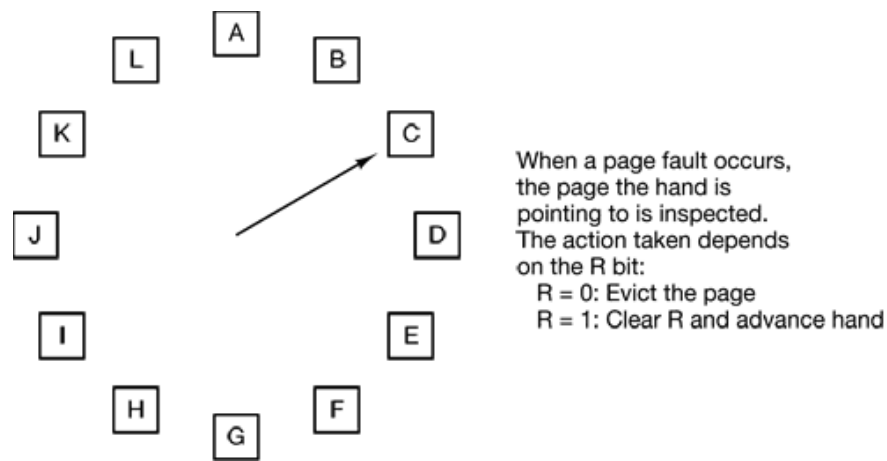


Figure 3-10 The clock page replacement algorithm.

- **Least Recently Used (LRU) Page Replacement Algorithm** –is an algorithm that , when a page fault occurs, throws out the page that has been unused for the longest time

Limitation

- To fully implement LRU, it is necessary to maintain a linked list of all pages in memory, with the most recently used page at the front and the least recently used page at the rear. The difficulty is that the list must be updated on every memory reference. Finding a page in the list, deleting it, and then moving it to the front is a very time consuming operation, even in hardware (assuming that such hardware could be built).
- **Working Set Page Replacement Algorithm** – finds a page that is not in the working set and evicts it. A **working set** is the set of pages that a process is currently using.
- **Working Set Clock Page Replacement Algorithm** -

Segmentation

The virtual memory discussed so far is one-dimensional because the virtual addresses go from 0 to some maximum address, one address after another. For many problems, having two or more separate virtual address spaces may be much better than having only one. For example, a compiler has many tables that are built up as compilation proceeds, possibly including:

- *The source text being saved for the printed listing (on batch systems).*
- *The symbol table, containing the names and attributes of variables.*
- *The table containing all the integer and floating-point constants used.*
- *The parse tree, containing the syntactic analysis of the program.*
- *The stack used for procedure calls within the compiler.*

Each of the first four tables grows continuously as compilation proceeds. The last one grows and shrinks in unpredictable ways during compilation. In a one-dimensional memory, these five tables would have to be allocated contiguous chunks of virtual address space, as in Fig. 3-11

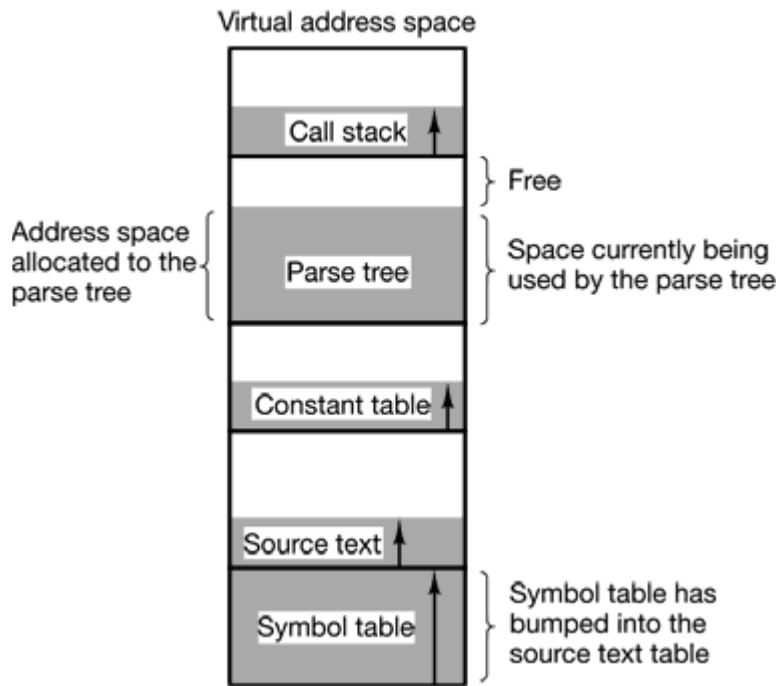


Figure 3-11 In a one-dimensional address space with growing tables, one table may bump into another.

Consider what happens if a program has an exceptionally large number of variables but a normal amount of everything else. The chunk of address space allocated for the symbol table may fill up, but there may be lots of room in the other tables. The compiler could, of course, simply issue a message saying that the compilation cannot continue due to too many variables, but doing so does not seem very sporting when unused space is left in the other tables.

Another possibility is to play Robin Hood, taking space from the tables with an excess of room and giving it to the tables with little room. This shuffling can be done, but it is analogous to managing one's own overlays—a nuisance at best and a great deal of tedious, unrewarding work at worst. What is really needed is a way of freeing the programmer from having to manage the expanding and contracting tables; in the same way that virtual memory eliminates the worry of organizing the program into overlays.

A straightforward and extremely general solution is to provide the machine with many completely independent address spaces, called segments. Each segment consists of a linear sequence of addresses, from 0 to some maximum. The length of each segment may be anything from 0 to the maximum allowed. Different segments may, and usually do, have different lengths. Moreover, segment lengths may change during execution. The length of a stack segment may be increased whenever something is pushed onto the stack and decreased whenever something is popped off the stack.

Because each segment constitutes a separate address space, different segments can grow or shrink independently, without affecting each other. If a stack in a certain segment needs more address space to grow, it can have it, because there is nothing else in its address space to bump into. Of course, a segment can fill up but segments are usually very large, so this occurrence is rare. To specify an address in this segmented or two-dimensional memory, the program must supply a two-part address, a segment number, and an address within the segment. Figure 3-12 illustrates a

segmented memory being used for the compiler tables discussed earlier. Five independent segments are shown here.

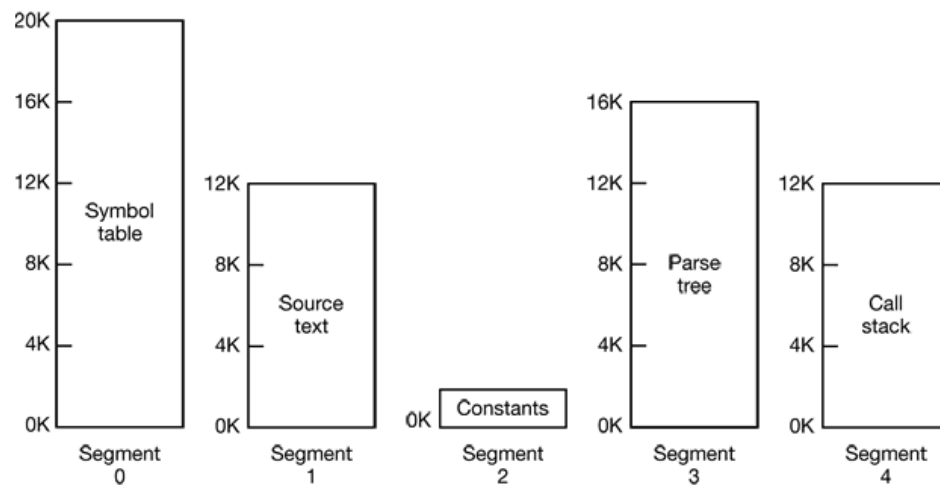


Figure 3-12. A segmented memory allows each table to grow or shrink independently of the other tables

A segment is a logical entity, which the programmer is aware of and uses as a logical entity. A segment might contain a procedure, or an array, or a stack, or a collection of scalar variables, but usually it does not contain a mixture of different types.

A segmented memory has other advantages besides simplifying the handling of data structures that are growing or shrinking. If each procedure occupies a separate segment, with address 0 in its starting address, the linking up of procedures compiled separately is greatly simplified. After all the procedures that constitute a program have been compiled and linked up, a procedure call to the procedure in .segment n will use the two-part address (n, 0) to address word 0 (the entry point).

If the procedure in segment n is subsequently modified and recompiled, no other procedures need be changed (because no starting addresses have been modified), even if the new version is larger than the old one. With a one-dimensional memory, the procedures are packed tightly next to each other, with no address space between them. Consequently, changing one procedure's size can affect the starting address of other, unrelated procedures. This, in turn, requires modifying all procedures that call any of the moved procedures, in order to incorporate their new starting addresses. If a program contains hundreds of procedures, this process can be costly.

Segmentation also facilitates sharing procedures or data between several processes. A common example is the **shared library**. Modern workstations that run advanced window systems often have extremely large graphical libraries compiled into nearly every program. In a segmented system, the graphical library can be put in a segment and shared by multiple processes, eliminating the need for having it in every process' address space. While it is also possible to have shared libraries in pure paging systems, it is much more complicated. In effect, these systems do it by simulating segmentation.

Because each segment forms a logical entity of which the programmer is aware, such as a procedure, or an array, or a stack, different segments can have different kinds of protection. A

procedure segment can be specified as execute only, prohibiting attempts to read from it or store into it. A floating-point array can be specified as read/write but not execute, and attempts to jump to it will be caught. Such protection is helpful in catching programming errors. Paging and segmentation are compared in Table 3-2.

The contents of a page are, in a sense, accidental. The programmer is unaware of the fact that paging is even occurring. Although putting a few bits in each entry of the page table to specify the access allowed would be possible, to utilize this feature the programmer would have to keep track of where in his address space the page boundaries were. That is precisely the sort of administration that paging was invented to eliminate. Because the user of a segmented memory has the illusion that all segments are in main memory all the time—that is, he can address them as though they were—he can protect each segment separately, without having to be concerned with the administration of overlaying them.

Table 3- 2 Comparison of paging and segmentation.

Consideration	Paging	Segmentation
Need the programmer be aware that this technique is being used?	No	Yes
How many linear address spaces are there?	1	Many
Can the total address space exceed the size of physical memory?	Yes	Yes
Can procedures and data be distinguished and separately protected?	No	Yes
Can tables whose size fluctuates be accommodated easily?	No	Yes
Is sharing of procedures between users facilitated?	No	Yes
Why was this technique invented?	To get a large linear address space without having to buy more physical memory	To allow programs and data to be broken up into logically independent address spaces and to aid sharing and protection

Implementation of Pure Segmentation

The implementation of segmentation differs from paging in an essential way: pages are fixed size and segments are not. Figure 3-12 (a) shows an example of physical memory initially containing five segments. Now consider what happens if segment 1 is evicted and segment 7, which is smaller, is put in its place. We arrive at the memory configuration of Fig. 3-12 (b). Between segment 7 and segment 2 is an unused area—that is, a hole. Then segment 4 is replaced by segment 5, as in Fig. 3-

12 (c), and segment 3 is replaced by segment 6, as in Fig. 3-12 (d). After the system has been running for a while, memory will be divided up into a number of chunks, some containing segments and some containing holes. This phenomenon, called **checkerboarding** or **external fragmentation**, wastes memory in the holes. It can be dealt with by compaction, as shown in Fig. 3-12 (e).

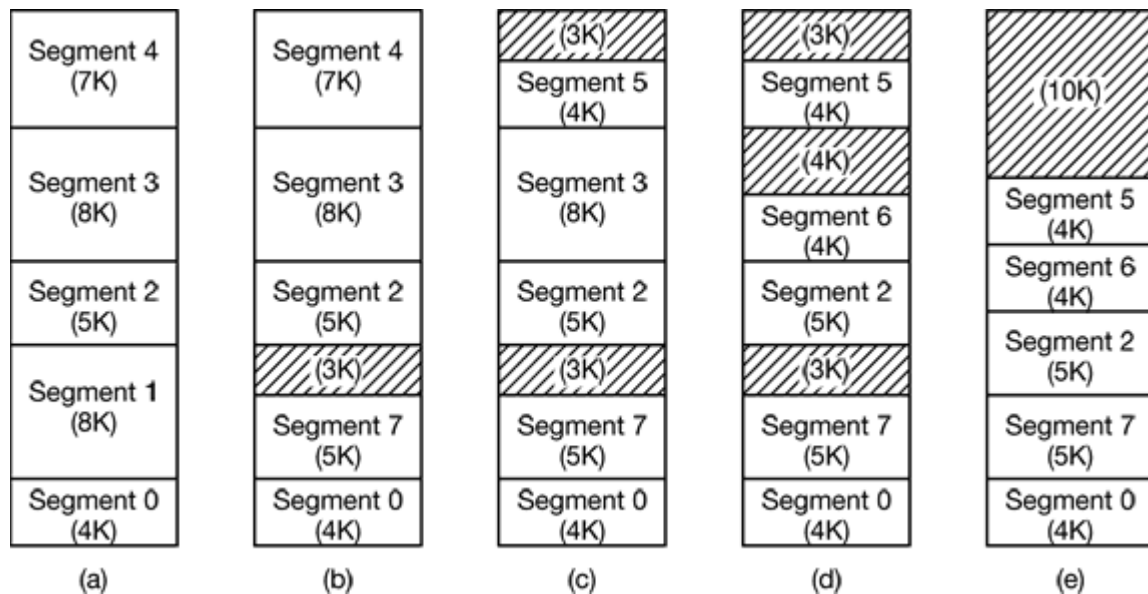


Figure 3-12. (a)-(d) Development of checkerboarding. (e) Removal of the checkerboarding by compaction.

If the segments are large, it may be inconvenient, or even impossible, to keep them in main memory in their entirety. This leads to the idea of paging them, so that only those pages that are actually needed have to be around. MULTICS was among the first systems to have supported paged segments and Intel Pentium is one of the more recent systems.

Segmentation with Paging: The Intel Pentium

The heart of the Pentium virtual memory consists of two tables, the **LDT (Local Descriptor Table)** and the **GDT (Global Descriptor Table)**. Each program has its own LDT, but there is a single GDT, shared by all the programs on the computer. The LDT describes segments local to each program, including its code, data, stack, and so on, whereas the GDT describes system segments, including the operating system itself.

To access a segment, a Pentium program first loads a selector for that segment into one of the machine's six segment registers. During execution, the CS register holds the selector for the code segment and the DS register holds the selector for the data segment. The other segment registers are less important. Each selector is a 16-bit number, as shown in Fig. 4-43.

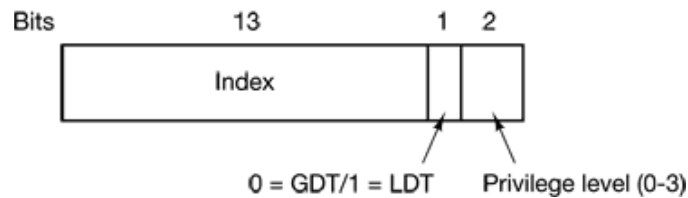


Figure 4-43. A Pentium selector.

One of the selector bits tells whether the segment is local or global (i.e., whether it is in the LDT or GDT). Thirteen other bits specify the LPT or GDT entry number, so these tables are each restricted to holding 8K segment descriptors. The other 2 bits relate to protection, and will be described later. Descriptor 0 is forbidden. It may be safely loaded into a segment register to indicate that the segment register is not currently available. It causes a trap it used.

At the time a selector is loaded into a segment register, the corresponding descriptor is fetched from the LDT or GDT and stored in microprogram registers, so it can be accessed quickly. A descriptor consists of 8 bytes, including the segment's base address, size, and other information, as depicted in Fig. 4-44.

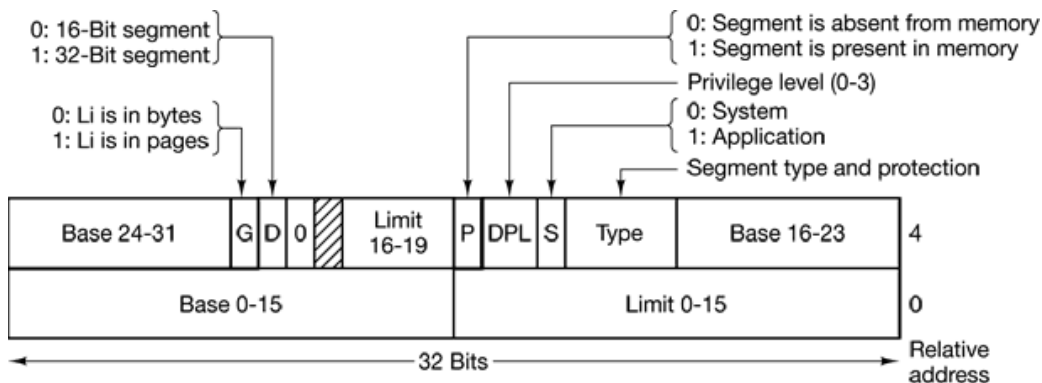


Figure 4-44. Pentium code segment descriptor. Data segments differ slightly.

The format of the selector has been cleverly chosen to make locating the descriptor easy. First either the LDT or GDT is selected, based on selector bit 2. Then the selector is copied to an internal scratch register and the 3 low-order bits set to 0. Finally, the address of either the LDT or GDT table is added to it, to give a direct pointer to the descriptor. For example, selector 72 refers to entry 9 in the GDT, which is located at address $\text{GDT} + 72$.

Let us trace the steps by which a (selector, offset) pair is converted to a physical address. As soon as the microprogram knows which segment register is being used, it can find the complete descriptor corresponding to that selector in its internal registers. If the segment does not exist (selector 0), or is currently paged out, a trap occurs.

It then checks to see if the offset is beyond the end of the segment, in which case a trap also occurs. Logically, there should simply be a 32-bit field in the descriptor giving the size of the segment, but there are only 20 bits available, so a different scheme is used. If the Gbit (Granularity) field is 0, the Limit field is the exact segment size, up to 1 MB. If it is 1, the Limit field gives the segment size in pages instead of bytes. The Pentium page size is fixed at 4 KB, so 20 bits are enough for segments up to 2^{32} bytes.

Assuming that the segment is in memory and the offset is in range, the Pentium then adds the 32-bit Base field in the descriptor to the offset to form what is called a **linear address**, as shown in Fig. 4-45. The Base field is broken up into three pieces and spread all over the descriptor for compatibility with the 286, in which the Base is only 24 bits. In effect, the Base field allows each segment to start at an arbitrary place within the 32-bit linear address space.

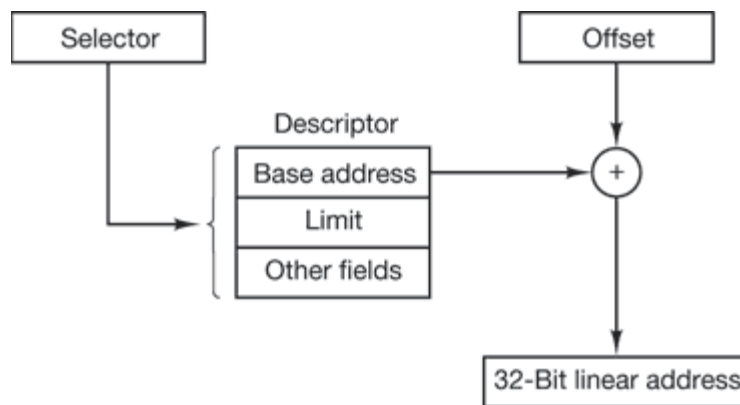


Figure 4-45. Conversion of a (selector, offset) pair to a linear address.

If paging is disabled (by a bit in a global control register), the linear address is interpreted as the physical address and sent to the memory for the read or write. Thus with paging disabled, we have a pure segmentation scheme, with each segment's base address given in its descriptor. Segments are permitted to overlap, incidentally, probably because it would be too much trouble and take too much time to verify that they were all disjoint.

On the other hand, if paging is enabled, the linear address is interpreted as a virtual address and mapped onto the physical address using page tables, pretty much as in our earlier examples. The only real complication is that with a 32-bit virtual address and a 4-KB page, a segment might contain 1 million pages, so a two-level mapping is used to reduce the page table size for small segments.

Each running program has a **page directory** consisting of 1024 32-bit entries. It is located at an address pointed to by a global register. Each entry in this directory points to a page table also containing 1024 32-bit entries. The page table entries point to page frames. The scheme is shown in Fig. 4-46.

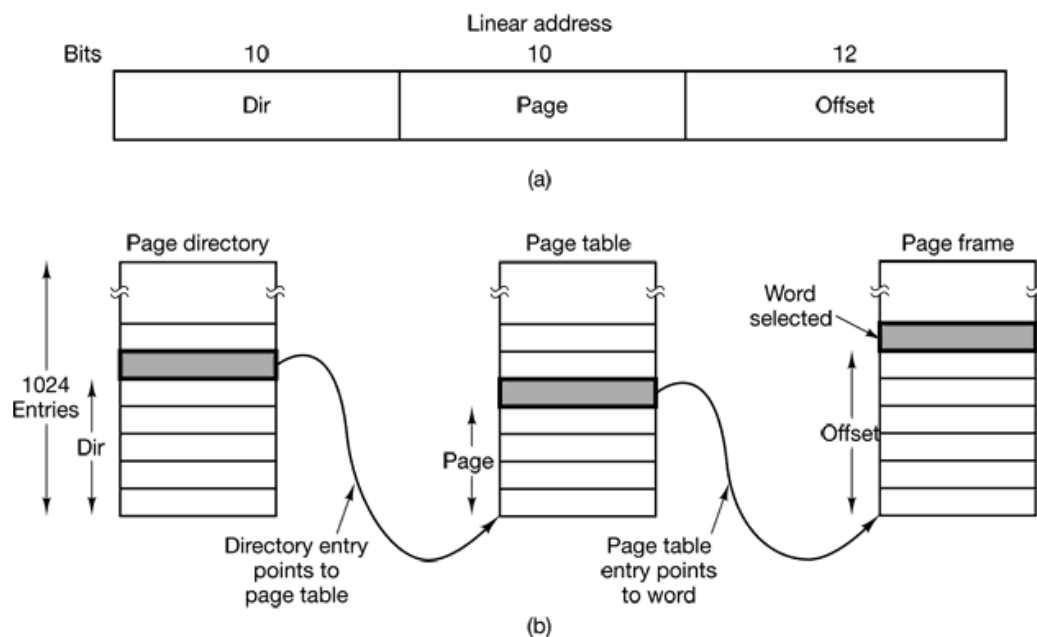


Figure 4-46. Mapping of a linear address onto a physical address.

In Fig. 4-46(a) we see a linear address divided into three fields. Dir, Page, and Offset. The Dir field is used to index into the page directory to locate a pointer to the proper page table. Then the Page field is used as an index into the page table to find the physical address of the page frame. Finally, Offset is added to the address of the page frame to get the physical address of the byte or word needed.

The page table entries are 32 bits each, 20 of which contain a page frame number. The remaining bits contain access and dirty bits, set by the hardware for the benefit of the operating system, protection bits, and other utility bits.

Each page table has entries for 1024 4-KB page frames, so a single page table handles 4 megabytes of memory. A segment shorter than 4M will have a page directory with a single entry, a pointer to its one and only page table. In this way, the overhead for short segments is only two pages, instead of the million pages that would be needed in a one-level page table.

To avoid making repeated references to memory, the Pentium has a small TLB that directly maps the most recently used Dir—Page combinations onto the physical address of the page frame. Only when the current combination is not present in the TLB is the mechanism of Fig. 4-46 actually carried out and the TLB updated. As long as TLB misses are rare, performance is good.

It is also worth noting that if some application does not need segmentation but is content with a single, paged, 32-bit address space, that model is possible. All the segment registers can be set up with the same selector, whose descriptor has Base = 0 and Limit set to the maximum. The instruction offset will then be the linear address, with only a single address space used—in effect, normal paging. In fact, all current operating systems for the Pentium work this way. OS/2 was the only one that used the full power of the Intel MMU architecture.

All in all, one has to give credit to the Pentium designers. Given the conflicting goals of implementing pure paging, pure segmentation, and paged segments, while at the same time being compatible with the 286, and doing all of this efficiently, the resulting design is surprisingly simple and clean.

Although we have covered the complete architecture of the Pentium virtual memory, albeit briefly, it is worth saying a few words about protection, since this subject is intimately related to the virtual memory. Just as the virtual memory scheme is closely modeled on MULTICS, so is the protection system. The Pentium supports four protection levels with level 0 being the most privileged and level 3 the least. These are shown in Fig. 4-47. At each instant, a running program is at a certain level, indicated by a 2-bit field in its PSW. Each segment in the system also has a level.

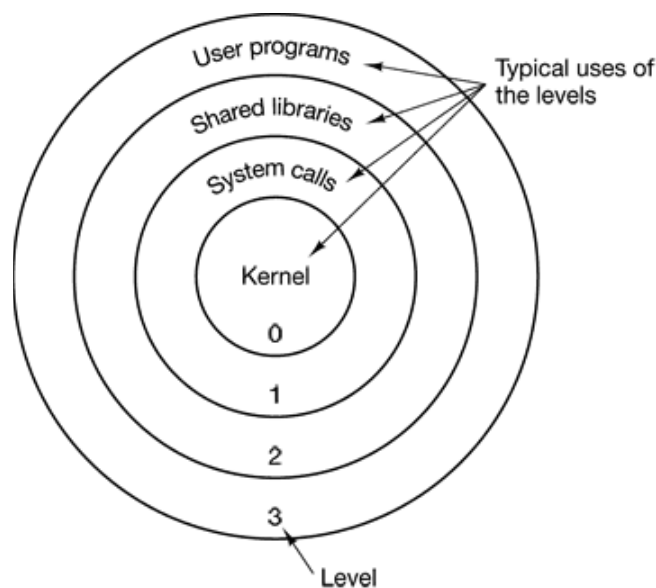


Figure 4-47. Protection on the Pentium

As long as a program restricts itself to using segments at its own level, everything works fine. Attempts to access data at a higher level are permitted. Attempts to access data at a lower level are

illegal and cause traps. Attempts to call procedures at a different level (higher or lower) are allowed, but in a carefully controlled way. To make an interlevel call, the CALL instruction must contain a selector instead of an address. This selector designates a descriptor called a **call gate**, which gives the address of the procedure to be called. Thus it is not possible to jump into the middle of an arbitrary code segment at a different level. Only official entry points may be used. The concepts of protection levels and call gates were pioneered in MULTICS, where they were viewed as **protection rings**.

A typical use for this mechanism is suggested in [Fig. 4-47](#). At level 0, we find the kernel of the operating system, which handles I/O, memory management, and other critical matters. At level 1, the system call handler is present. User programs may call procedures here to have system calls carried out, but only a specific and protected list of procedures may be called. Level 2 contains library procedures, possibly shared among many running programs. User programs may call these procedures and read their data, but they may not modify them. Finally, user programs run at level 3, which has the least protection.

Traps and interrupts use a mechanism similar to the call gates. They, too, reference descriptors, rather than absolute addresses, and these descriptors point to specific procedures to be executed. The Type field in Fig. 4-44 distinguishes between code segments, data segments, and the various kinds of gates.

Modeling Multiprogramming

When multiprogramming is used, the CPU utilization can be improved. A good way to look at CPU usage is from a probabilistic viewpoint. Suppose that a process spends a fraction p of its time waiting for I/O to complete. With n processes in memory at once, the probability that all n processes are waiting for I/O (in which case the CPU will be idle) is p^n . The CPU utilization is then given by the formula:

$$\text{CPU utilization} = 1 - p^n$$

Figure 3-4 shows the CPU utilization as a function of n , which is called the **degree of multiprogramming**.

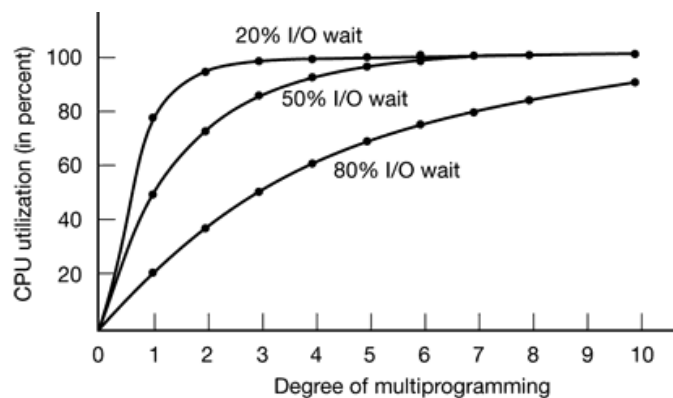


Figure 3-4. CPU utilization as a function of the number of processes in memory

From the figure it is clear that if processes spend 80 percent of their time waiting for I/O, at least 10 processes must be in memory at once to get the CPU waste below 10 percent. When you realize that an interactive process waiting for a user to type something at a terminal is in I/O wait state, it should be clear that I/O wait times of 80 percent and more are not unusual. But even in batch systems, processes doing a lot of disk I/O will often have this percentage or more.

It should be pointed out that the probabilistic model just described is only an approximation. It implicitly assumes that all n processes are independent, which is not true always.

Even though the model of Fig. 3-4 is simple-minded, it can nevertheless be used to make specific, although approximate, predictions about CPU performance. Suppose that a computer has 32 MB of memory, with the operating system taking up 16 MB and each user program taking up 4 MB. These sizes allow **four** user programs to be in memory at once. With an 80 percent average I/O wait, we have a CPU utilization (ignoring operating system overhead) of $1 - 0.8^4$ or about 60 percent.

Adding another 16 MB of memory allows the system to go from four-way multiprogramming to eight-way multiprogramming, thus raising the CPU utilization to 83 percent. In other words, the additional 16 MB will raise the throughput by 38 percent.

Adding yet another 16 MB would only increase CPU utilization from 83 percent to 93 percent, thus raising the throughput by only another 12 percent. Using this model the computer's owner might decide that the first addition is a good investment but that the second is not.

Calculation of throughput

PROBLEMS

1. A computer system has enough room to hold four programs in its main memory. These programs are idle waiting for I/O half the time. What fraction of the CPU time is wasted?
2. In Fig. 4-4 we saw an example of how multiple jobs can run in parallel and finish faster than if they had run sequentially. Suppose that two jobs, each of which needs 10 minutes of CPU time, start simultaneously. How long will the last one take to complete if they run sequentially? How long if they run in parallel? Assume 50% I/O wait.
3. A swapping system eliminates holes by compaction. Assuming a random distribution of many holes and many data segments and a time to read or write a 32-bit memory word of 10 nsec, about how long does it take to compact 128 MB? For simplicity, assume that word 0 is part of a hole and that the highest word in memory contains valid data.
4. In this problem you are to compare the storage needed to keep track of free memory using a bitmap versus using a linked list. The 128-MB memory is allocated in units of n bytes. For the linked list, assume that memory consists of an alternating sequence of segments and holes, each 64 KB. Also assume that each node in the linked list needs a 32-bit memory address, a 16-bit length, and a 16-bit next-node field. How many bytes of storage is required for each method? Which one is better?
5. Consider a swapping system in which memory consists of the following hole sizes in memory order: 10 KB, 4 KB, 20 KB, 18 KB, 7 KB, 9 KB, 12 KB, and 15 KB. Which hole is taken for successive segment requests of
 - (a) 12 KB
 - (b) 10 KB
 - (c) 9 KBfor first fit? Now repeat the question for best fit, worst fit, and next fit.
6. What is the difference between a physical address and a virtual address?
7. For each of the following decimal virtual addresses, compute the virtual page number and offset for a 4-KB page and for an 8 KB page: 20000, 32768, 60000.
8. Using the page table of Fig. 4-10, give the physical address corresponding to each of the following virtual addresses:
 - (a) 20
 - (b) 4100
 - (c) 8300
9. The Intel 8086 processor does not support virtual memory. Nevertheless, some companies previously sold systems that contained an unmodified 8086 CPU and do paging. Make an educated guess as to how they did it. Hint: Think about the logical location of the MMU.
10. The amount of disk space that must be available for page storage is related to the maximum number of processes, n , the number of bytes in the virtual address space, v , and the number of bytes of RAM, r . Give an expression for the worst case disk space requirements. How realistic is this amount?
11. If an instruction takes 10 nsec and a page fault takes an additional n nsec, give a formula for the effective instruction time if page faults occur every k instructions.
12. A machine has a 32-bit address space and an 8-KB page. The page table is entirely in hardware, with one 32-bit word per entry. When a process starts, the page table is copied to the hardware from memory, at one word every 100 nsec. If each process runs for 100 msec (including the time to load the page table), what fraction of the CPU time is devoted to loading the page tables?

13. A computer with a 32-bit address uses a two-level page table. Virtual addresses are split into a 9-bit top-level page table field, an 11-bit second-level page table field, and an offset. How large are the pages and how many are there in the address space?
14. Suppose that a 32-bit virtual address is broken up into four fields, a, b, c, and d. The first three are used for a three-level page table system. The fourth field, d, is the offset. Does the number of pages depend on the sizes of all four fields? If not, which ones matter and which ones do not?
15. A computer has 32-bit virtual addresses and 4KB pages. The program and data together fit in the lowest page (0–4095). The stack fits in the highest page. How many entries are needed in the page table if traditional (one-level) paging is used? How many page table entries are needed for two-level paging, with 10 bits in each part?
16. Below is an execution trace of a program fragment for a computer with 512-byte pages. The program is located at address 1020, and its stack pointer is at 8102 (the stack grows toward 0). Give the page reference string generated by this program. Each instruction occupies 4 bytes (1 word) including immediate constants. Both instruction and data references count in the reference string.

Load word 6144 into register 0
 Push register 0 onto the stack
 Call a procedure at 5120, stacking the return address
 Subtract the immediate constant 16 from the stack pointer
 Compare the actual parameter to the immediate constant 4
 Jump if equal to 5152

17. A computer whose processes have 1024 pages in their address spaces keeps its page tables in memory. The overhead required for reading a word from the page table is 5 nsec. To reduce this overhead, the computer has a TLB, which holds 32 (virtual page, physical page frame) pairs, and can do a look up in 1 nsec. What hit rate is needed to reduce the mean overhead to 2 nsec?
18. The TLB on the VAX does not contain an R bit. Why?
19. How can the associative memory device needed for a TLB be implemented in hardware, and what are the implications of such a design for expandability?
20. A machine has 48-bit virtual addresses and 32-bit physical addresses. Pages are 8 KB. How many entries are needed for the page table?
21. A computer with an 8-KB page, a 256-KB main memory, and a 64-GB virtual address space uses an inverted page table to implement its virtual memory. How big should the hash table be to ensure a mean hash chain length of less than 1? Assume that the hash table size is a power of two.

A student in a compiler design course proposes to the professor a project of writing a compiler that will produce a list of page references that can be used to implement the optimal page replacement algorithm. Is this possible? Why or why not? Is there anything that could be done to improve paging efficiency at run time?

If FIFO page replacement is used with four page frames and eight pages, how many page faults will occur with the reference string 0172327103 if the four frames are initially empty? Now repeat this problem for LRU.

Consider the page sequence of Fig. 4-16(b). Suppose that the R bits for the pages B through A are 11011011, respectively. Which page will second chance remove?

A small computer has four page frames. At the first clock tick, the R bits are 0111 (page 0 is 0, the rest are 1). At subsequent clock ticks, the values are 1011, 1010, 1101, 0010, 1010, 1100, and 0001. If the aging algorithm is used with an 8-bit counter, give the values of the four counters after the last tick.

Suppose that $\tau = 400$ in Fig. 4-21. Which page will be removed?

In the WSClock algorithm of Fig. 4-22(c), the hand points to a page with $R = 0$. If $\tau = 400$, will this page be removed? What about if $\tau = 1000$?

How long does it take to load a 64-KB program from a disk whose average seek time is 10 msec, whose rotation time is 10 msec, and whose tracks hold 32 KB

(a) for a 2-KB page size?

(b) for a 4-KB page size?

The pages are spread randomly around the disk and the number of cylinders is so large that the chance of two pages being on the same cylinder is negligible.

A computer has four page frames. The time of loading, time of last access, and the R and M bits for each page are as shown below (the times are in clock ticks):

Page	Loaded	Last ref.	R	M
0	126	280	1	0
1	230	265	0	01
2	140	270	0	0
3	110	285	1	1

(a) Which page will NRU replace?

(b) Which page will FIFO replace?

(c) Which page will LRU replace?

(d) Which page will second chance replace?

One of the first timesharing machines, the PDP-1, had a memory of 4K 18-bit words. It held one process at a time in memory. When the scheduler decided to run another process, the process in memory was written to a paging drum, with 4K 18-bit words around the circumference of the drum. The drum could start writing (or reading) at any word, rather than only at word 0. Why do you suppose this drum was chosen?

A computer provides each process with 65,536 bytes of address space divided into pages of 4096 bytes. A particular program has a text size of 32,768 bytes, a data size of 16,386 bytes, and a stack size of 15,870 bytes. Will this program fit in the address space? If the page size were 512 bytes, would it fit? Remember that a page may not contain parts of two different segments.

Can a page be in two working sets at the same time? Explain.

If a page is shared between two processes, is it possible that the page is read-only for one process and read-write for the other? Why or why not?

It has been observed that the number of instructions executed between page faults is directly proportional to the number of page frames allocated to a program. If the available memory is doubled, the mean interval between page faults is also doubled. Suppose that a normal instruction takes 1 microsec, but if a page fault occurs, it takes 2001 μ sec (i.e., 2 msec to handle the fault). If a program takes 60 sec to run, during which time it gets 15,000 page faults, how long would it take to run if twice as much memory were available?

A group of operating system designers for the Frugal Computer Company are thinking about ways of reducing the amount of backing store needed in their new operating system. The head guru has just suggested not bothering to save the program text in the swap area at all, but just page it in directly from the binary file whenever it is needed. Under what conditions, if any, does this idea work for the program text? Under what conditions, if any, does it work for the data?

A machine language instruction to load a 32-bit word into a register contains the 32-bit address of the word to be loaded. What is the maximum number of page faults this instruction can cause?

Explain the difference between internal fragmentation and external fragmentation. Which one occurs in paging systems? Which one occurs in systems using pure segmentation?

When segmentation and paging are both being used, as in MULTICS, first the segment descriptor must be looked up, then the page descriptor. Does the TLB also work this way, with two levels of lookup?

Plot a histogram and calculate the mean and median of the sizes of executable binary files on a computer to which you have access. On a Windows system, look at all .exe and .dll files; on a UNIX system look at all executable files in /bin, /usr/bin, and /local/bin that are not scripts (or use the file utility to find all executables). Determine the optimal page size for this computer just considering the code (not data). Consider internal fragmentation and page table size making some reasonable assumption about the size of a page table entry. Assume that all programs are equally likely to be run and thus should be weighted equally.

Small programs for MS-DOS can be compiled as .COM files. These files are always loaded at address 0x100 in a single memory segment that is used for code, data, and stack. Instructions that transfer control of execution, such as JMP and CALL, or that access static data from fixed addresses have the addresses compiled into the object code. Write a program that can relocate such a program file to run starting at an arbitrary address. Your program must scan through code looking for object codes for instructions that refer to fixed memory addresses, then modify those addresses that point to memory locations within the range to be relocated. You can find the object codes in an assembly language programming text. Note that doing this perfectly without additional information is, in general, an impossible task, because some data words may have values that mimic instruction object codes.

Write a program that simulates a paging system. At the start of the program, the user should be asked to choose a page replacement algorithm, choosing from FIFO, LRU, and at least one other. On each cycle, read the number of the referenced page from a file. Generate a listing similar to Fig. 4-25, except rotated 90 degrees so that each new page reference increases the length of the output file by one line.

Write a program that models the distance string algorithm described in the text. The input to the program is a list of page references (contained in a file), plus the number of page frames of physical memory available. If possible, use trace data from real programs instead of randomly-generated page references. The program should maintain the stack of pages, analogous to Fig. 4-25. At each page fault, a procedure should be called to choose a replacement page. When the run is completed, the program should plot the distance string, analogous to Fig. 4-26. Make multiple runs for different values of the memory size and see what conclusions you can draw.